

Primary sources & further reading

Track 16.5 — 🌱 LangSmith · 16-ai-tech-stack/tracks/langsmith/papers/PAPERS.md · 16-ai-tech-stack/tracks/langsmith/resources/RESOURCES.md



Primary Sources — LangSmith, Tracing & LLM Evaluation

LangSmith is a product, not a research field, so the primary sources here are **official docs** first and papers second. Every doc URL below was verified in September 2026. Where a deep path may move, the section root is given instead.

Read these three first

#	Source	Why it matters	Link
1	LangSmith observability concepts	The exact definitions of run, trace, thread, trajectory, project, feedback, tag, metadata. Read it once and the UI stops being mysterious.	https://docs.langchain.com/langsmith/observability-concepts
2	LangSmith evaluation concepts	Datasets, examples, splits, dataset versions, experiments, evaluators, LLM-as-judge, pairwise, offline vs online evaluation. The vocabulary for the whole second half of this track.	https://docs.langchain.com/langsmith/evaluation-concepts
3	Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena — Zheng et al., 2023	The paper that named position bias, verbosity bias and self-enhancement bias, and measured them. Everything in §3 of note 02 traces back here.	arXiv:2306.05685

Official LangSmith documentation

Topic	What to take from it	Link
Observability quickstart	The env vars, <code>@traceable</code> , <code>wrap_openai</code> — the minimum viable instrumentation	https://docs.langchain.com/langsmith/observability-quickstart
Trace sampling	<code>LANGSMITH_TRACING_SAMPLING_RATE</code> , 0.0–1.0, applied at the root	https://docs.langchain.com/langsmith/sample-traces
Mask inputs and outputs	The four PII seams: <code>process_inputs</code> , <code>create_anonymizer</code> , <code>hide_inputs</code> , <code>LANGSMITH_HIDE_*</code>	https://docs.langchain.com/langsmith/mask-inputs-outputs
Evaluate an LLM application	The <code>evaluate()</code> / <code>aevaluate()</code> signature and the evaluator contract	https://docs.langchain.com/langsmith/evaluate-llm-application
Pairwise evaluation	<code>evaluate_comparative()</code> and <code>randomize_order</code> as position-bias mitigation	https://docs.langchain.com/langsmith/evaluate-pairwise
Manage datasets	<code>create_dataset</code> , <code>create_examples</code> , splits, automatic versions, <code>update_dataset_tag</code>	https://docs.langchain.com/langsmith/manage-datasets
Annotation queues	Rubrics, reviewer config, multiple reviewers, reservations, pairwise queues, Add to Dataset	https://docs.langchain.com/langsmith/annotation-queues
Online evaluations	Attaching judges to live traffic: filters, sampling rate, spend limits, backfill	https://docs.langchain.com/langsmith/online-evaluations
Automations and rules	Routing runs to datasets, queues and webhooks automatically	https://docs.langchain.com/langsmith/rules
Dashboards and alerts	What to monitor, and how alerting is wired	https://docs.langchain.com/langsmith/dashboards
Prompt engineering concepts	Commits, commit tags, reserved <code>staging/production</code> tags, <code>pull_prompt</code>	https://docs.langchain.com/langsmith/prompt-engineering-concepts
Trace with OpenTelemetry	OTLP ingest and export — the escape hatch that makes this a reversible bet	https://docs.langchain.com/langsmith/trace-with-opentelemetry
Platform setup	Cloud / BYOC / self-hosted, and which plan gates which	https://docs.langchain.com/langsmith/platform-setup
LangSmith Engine	2026 feature: mines traces for recurring failure patterns, proposes fixes and datasets	https://docs.langchain.com/langsmith/engine-overview
Python SDK reference	The authoritative signatures when the guides disagree	https://reference.langchain.com/python/langsmith
<code>langsmith-sdk</code> on GitHub	Read <code>run_helpers.py</code> — it is the real version of <code>minimal_tracer.py</code>	https://github.com/langchain-ai/langsmith-sdk

Papers worth the time

Paper	Year	One-line takeaway	Link
Judging LLM-as-a-Judge (MT-Bench / Chatbot Arena) — Zheng et al.	2023	Judge biases, named and measured; ~80% agreement with humans at best	arXiv:2306.05685
G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment — Liu et al.	2023	Chain-of-thought + form-filling judging; the template most rubric judges descend from	arXiv:2303.16634
Large Language Models are not Fair Evaluators — Wang et al.	2023	Position bias quantified; swapping candidate order flips verdicts	arXiv:2305.17926
Benchmarking Cognitive Biases in LLMs as Evaluators (CoBBLEr)	2023	Six evaluator biases beyond position and verbosity	arXiv:2309.17012
Hidden Technical Debt in Machine Learning Systems — Sculley et al.	2015	The model is 5% of the system; monitoring and data debt are the rest. The intellectual ancestor of this entire track.	PDF
Dapper, a Large-Scale Distributed Systems Tracing Infrastructure — Sigelman et al.	2010	Where spans, parent ids and trace sampling come from. LangSmith's data model is Dapper with token counts.	PDF
Constitutional AI: Harmlessness from AI Feedback — Bai et al.	2022	The origin of "a model can score another model's output against written criteria"	arXiv:2212.08073

Standards and neighbours

Source	Why	Link
OpenTelemetry GenAI semantic conventions	The vendor-neutral span/attribute spec. Instrument to this and your backend becomes a choice.	https://github.com/open-telemetry/semantic-conventions-genai
OpenTelemetry tracing specification	Spans, context propagation, samplers — the concepts LangSmith renames	https://opentelemetry.io/docs/concepts/signals/traces/
NIST AI Risk Management Framework	The "Measure" and "Manage" functions are this track, in policy language	https://www.nist.gov/itl/ai-risk-management-framework

How to read a vendor doc without getting sold

1. **Find the data model page first.** Products are easy once you know their nouns. Skip the tutorials.
2. **Then find the limits page.** Retention, rate limits, trace size caps, plan gating. This is where the architecture constraints hide, and it is never in the tutorial.
3. **Then find the escape hatch.** Export, OTLP, self-hosting. If there isn't one, price that in.
4. **Only then read a quickstart.** By that point you are checking syntax, not learning concepts.

Resources — LangSmith, Tracing & LLM Observability

Start here

Resource	Who it's for	Link
LangSmith docs home	The canonical entry point; the left nav is the product map	https://docs.langchain.com/langsmith/home
LangSmith app	Free tier is enough for everything in ../lab/EXERCISES.md §7	https://smith.langchain.com/
LangChain Academy	Free courses; the LangSmith and LangGraph ones pair with this track	https://academy.langchain.com/
LangChain blog	Where new LangSmith capabilities are announced before the docs settle	https://blog.langchain.com/

Repositories worth cloning

Repo	What's inside
https://github.com/langchain-ai/langsmith-sdk	The Python + JS SDKs. Read <code>python/langsmith/run_helpers.py</code> — it is the production version of <code>../code/minimal_tracer.py</code>
https://github.com/langchain-ai/openevals	Prebuilt LLM-as-judge evaluators (correctness, groundedness, conciseness) you can read and steal the prompts from
https://github.com/langchain-ai/langsmith-cookbook	Task-shaped recipes: CI evals, feedback plumbing, dataset construction
https://github.com/langfuse/langfuse	The main open-source alternative. Self-hostable; worth running once to compare data models
https://github.com/Arize-ai/phoenix	OTel-native open-source tracing + eval; runs in a notebook
https://github.com/open-telemetry/semantic-conventions-genai	The vendor-neutral GenAI span/attribute spec
https://github.com/traceloop/openllmetry	OTel instrumentation for LLM libraries; the format LangSmith also accepts
https://github.com/confident-ai/deepeval	Pytest-style LLM evals — the "evals as unit tests" ergonomic, if that is your taste
https://github.com/explodinggradients/ragas	RAG-specific metrics (faithfulness, context precision/recall) that plug into an eval harness

Alternatives, first-party docs

Read at least two of these before you commit to one vendor. Their data models differ in ways that matter.

Tool	Link
Langfuse	https://langfuse.com/docs
Arize Phoenix	https://arize.com/docs/phoenix
Braintrust	https://www.braintrust.dev/docs
Weights & Biases Weave	https://weave-docs.wandb.ai/
Helicone	https://docs.helicone.ai/
OpenTelemetry (tracing concepts)	https://opentelemetry.io/docs/concepts/signals/traces/

Adjacent tracks in this repo

Where	Why you'd go there
../15-ai-evals/	The theory this track is the tooling for. Read it.
../langchain/ · ../langgraph/	The frameworks that trace automatically
../08-rag/	Where most "hallucinations" in a traced system actually come from
../06-ai-agents/ · ../07-agentic-ai/	Multi-step loops are the reason a tree view beats a log
../13-ai-security/	PII redaction, adversarial eval sets
../12-ai-governance/ · ../14-ai-compliance/	Traces and experiments as audit evidence
../11-llm-models/	Model migration is a pairwise-eval problem

Cheat sheet — environment variables

```

LANGSMITH_TRACING=true           # master switch; forget this and nothing appears
LANGSMITH_API_KEY=lsv2_pt...
LANGSMITH_PROJECT=my-app-prod    # default tracing project
LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com # data region (EU shown)
LANGSMITH_TRACING_SAMPLING_RATE=0.1 # 0.0-1.0, applied at the trace root
LANGSMITH_HIDE_INPUTS=true       # blunt PII kill-switch
LANGSMITH_HIDE_OUTPUTS=true
LANGSMITH_HIDE_METADATA=true

```

Communities

- LangChain Forum — <https://forum.langchain.com/>
- LangChain on GitHub Discussions — <https://github.com/langchain-ai/langchain/discussions>
- r/LLMDevs — <https://www.reddit.com/r/LLMDevs/>
- OpenTelemetry community — <https://opentelemetry.io/community/>

Also see

- [../_shared/cheatsheets/](#)